SQL Fundamentals: DDL, Constraints, DML and Security

Prof. Vincenzo Moscato

Dott. Antonio Laudante

Dott. Francesco Di Serio

SQL Overview

SQL (Structured Query Language) is the standard language for relational DBMS.

It is a declarative language: users specify what they want, not how.

DDL – Data Definition Language

DML – Data Manipulation Language

Users, roles and privileges

Structure of a Relational Database

- Data are organized into **relations** (tables).
- A relation is a set of **tuples** (rows) over defined domains.
- Key properties:
  - ❑ tuples are unique
  - ❑ order of tuples is irrelevant
  - ❑ each attribute belongs to a domain
- Example: STUDENTS(<u>StudentID</u>, FirstName, LastName, Address)

Data Definition Language (DDL)

- DDL defines the **structure** of a database schema.
- Main commands:
  - **CREATE**
  - **ALTER**
  - **DROP**
- Schema definitions are stored in the database catalog.

CREATE TABLE

- **CREATE TABLE** defines the schema of a relation.
- Syntax:

```
CREATE TABLE TableName (
    Attribute1 Domain [Constraints],
    Attribute2 Domain [Constraints],
    Attribute3 Domain [Constraints],
    …
);
```

- Each attribute requires a name, type, and optional constraints.

EXAMPLE – CREATE TABLE

```
CREATE TABLE STUDENTS (
    StudentID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Address VARCHAR(100)
);
```

ALTER TABLE

- **ALTER TABLE** modifies an existing table.

- Typical operations:
  - ❑ **add** columns
  - ❑ **remove** columns
  - ❑ **add** constraints
  - ❑ **rename** columns

EXAMPLES – ALTER TABLE

Adding an attribute:

```
ALTER TABLE STUDENTS
ADD Email VARCHAR(100);
```

Removing a column:

```
ALTER TABLE STUDENTS
DROP COLUMN Address;
```

Adding a constraint:

```
ALTER TABLE STUDENTS
ADD CONSTRAINT unique_email
UNIQUE (Email);
```

Renaming a column:

```
ALTER TABLE STUDENTS
RENAME COLUMN FirstName TO
Name;
```

DROP TABLE

- **DROP TABLE** removes a table from the database.
- Syntax:

```
DROP TABLE TableName [RESTRICT | CASCADE];
```

- RESTRICT: allowed only if no dependencies exist.
- CASCADE: dependent objects are removed.

Integrity Constraints

Integrity constraints ensure database **consistency**.

They enforce **rules** that must always hold.

If all constraints are ***satisfied*** → database instance is ***legal***.

They can be divided into two main categories: **intra-relational constraints** and **inter-relational constraints**.

Intra-relational Constraints

1. Domain constraints

Restrict the values that an attribute can assume.

Examples:

- attribute cannot be NULL
- value must belong to a specific range

```
CREATE TABLE EXAMS (
   Grade INT CHECK (Grade BETWEEN 18 AND 30),
   StudentID INT NOT NULL
);
```

Intra-relational Constraints

2. Tuple constraints

Rules involving **multiple attributes** of the same tuple.

Example:

"Lode = YES only if Grade = 30"

```
CREATE TABLE EXAMS (
   Grade INT CHECK (Grade BETWEEN 18 AND 30),
   StudentID INT NOT NULL,
   Lode VARCHAR2(3),
   CONSTRAINT chk_lode CHECK ((Lode = 'YES' AND Grade = 30) OR Lode = 'NO')
);
```

Intra-relational Constraints

3. Key constraints

Ensure that each tuple is **uniquely identifiable**.

- A **key** is a minimal set of attributes that uniquely identifies a row.
- The **Primary Key** uniquely identifies each tuple, and cannot contain **NULL values.**
- Example:
  "StudentID" in table STUDENT.

Inter-relational Constraints

- Since relational databases distribute information across multiple tables to avoid redundancy, **inter-relational constraints** maintain **consistency** between tables.
- The most important inter-relational constraint is the **Referential Integrity**, or **Foreign Key**.
- A **foreign key** links two tables.
- It ensures that every value in the foreign key **exists** in the referenced primary key.

EXAMPLE – FOREIGN KEY

```
CREATE TABLE STUDENTS (
   StudentID INT PRIMARY KEY,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50) NOT NULL,
   Address VARCHAR(100)
);

CREATE TABLE EXAMS (
   Grade INT CHECK (Grade BETWEEN 18 AND 30),
   StudentID INT NOT NULL,
   CONSTRAINT FK_STUDENT FOREIGN KEY (StudentID)
     REFERENCES STUDENTS(StudentID)
);
```

Data Manipulation Language (DML)

- DML allows **modification** and **retrieval** of data.
- Main commands:
  - **INSERT**
  - **UPDATE**
  - **DELETE**
  - **SELECT**
- DML operations may belong to transactions.

INSERT

- **INSERT** adds new tuples into a table.
- Syntax:

```
INSERT INTO TableName VALUES(...);
```

- Examples:

```
INSERT INTO STUDENTS VALUES (1, 'Francesco', 'Di Serio',
'Naples');

INSERT INTO STUDENTS VALUES (2, 'Antonio', 'Laudante',
'Naples');
```

UPDATE

- **UPDATE** modifies existing tuples.
- Syntax:

```
UPDATE TableName
SET attribute=value
WHERE condition;
```

- Example:

```
UPDATE STUDENTS
SET Address = 'New York'
WHERE StudentID = 1;
```

DELETE

- **DELETE** removes tuples from a table.
- Syntax:

```
DELETE FROM TableName
WHERE condition;
```

- If WHERE is omitted → **all rows** are deleted.
- Example:

```
DELETE FROM STUDENTS
WHERE StudentID = 2;
```

Database Security

- **DBMS** (Data Base Management Systems) provide mechanisms to **protect** data.
- Concepts:
  - Users
  - Authentication
  - Privileges
  - Roles
- The **DBA** (Data Base Administrator) manages database security.

Creating Users

- Creating a **user** allows the system to uniquely identify and authenticate anyone accessing the database, enabling the assignment of specific credentials and privileges to **securely** operate on the system.
- Syntax:

```
CREATE USER username IDENTIFIED BY password;
```

- **Privileges** specify allowed operations (SELECT, INSERT, UPDATE, DELETE). Syntax:

```
GRANT PrivilegeList ON TableName TO username;
```

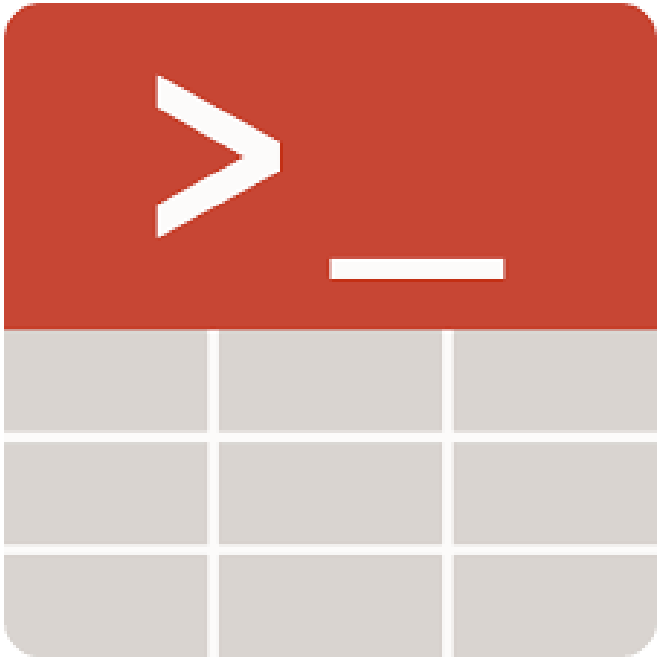
Roles and Access Management

- Creating a **role** allows **multiple privileges** to be grouped into a single profile. Users assigned to the role automatically inherit those permissions, **simplifying security management**.
- Syntax:

```
CREATE ROLE RoleName;

GRANT PrivilegeList ON TableName to RoleName;

Grant RoleName to username;
```

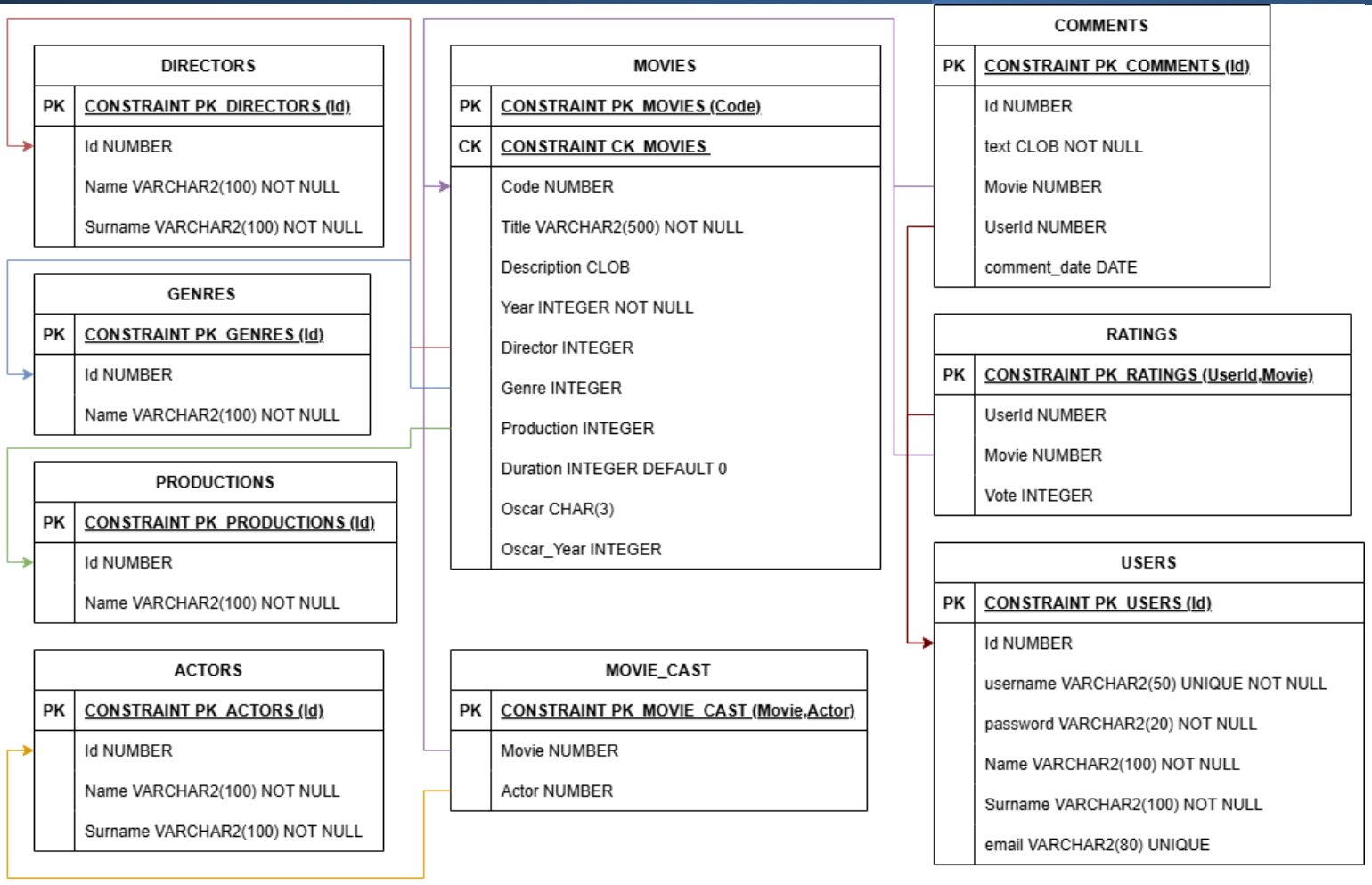


Practice Exercises

Oracle FreeSQL

https://freesql.com/

The Movies Database



The Movies Database

```
CREATE TABLE MOVIES(
   Code NUMBER,
   Title VARCHAR2(500) NOT NULL,
   Description CLOB,
   Year INTEGER CHECK(Year>1900) NOT NULL,
   Director INTEGER,
   Genre INTEGER,
   Production INTEGER,
   Duration INTEGER DEFAULT 0,
   Oscar CHAR(3) CHECK(Oscar='YES' OR Oscar='NO'),
   Oscar_Year INTEGER CHECK(Oscar_Year>1900),
   CONSTRAINT PK_MOVIES PRIMARY KEY(Code),
   CONSTRAINT CK_MOVIES CHECK((Oscar='YES' AND Oscar_Year IS NOT NULL)
      OR (Oscar='NO' AND Oscar_Year IS NULL))
);
```

The Movies Database

```
CREATE TABLE GENRES (
   Id NUMBER,
   Name VARCHAR2(100) NOT NULL,
   CONSTRAINT PK_GENRES PRIMARY KEY(Id)
);

CREATE TABLE DIRECTORS (
   Id NUMBER,
   Name VARCHAR2(100) NOT NULL,
   Surname VARCHAR2(100) NOT NULL,
   CONSTRAINT PK_DIRECTORS PRIMARY KEY(Id)
);
```

The Movies Database

```
CREATE TABLE ACTORS (
   Id NUMBER,
   Name VARCHAR2(100) NOT NULL,
   Surname VARCHAR2(100) NOT NULL,
   CONSTRAINT PK_ACTORS PRIMARY KEY(Id)
);

CREATE TABLE PRODUCTIONS (
   Id NUMBER,
   Name VARCHAR2(100) NOT NULL,
   CONSTRAINT PK_PRODUCTIONS PRIMARY KEY(Id)
);
```

The Movies Database

```
CREATE TABLE MOVIE_CAST (
   Movie NUMBER,
   Actor NUMBER,
   CONSTRAINT PK_MOVIE_CAST PRIMARY KEY(Movie,Actor)
);

CREATE TABLE USERS (
   Id NUMBER,
   username VARCHAR2(50) UNIQUE NOT NULL,
   password VARCHAR2(20) CHECK(LENGTH(password)>=5) NOT NULL,
   Name VARCHAR2(100) NOT NULL,
   Surname VARCHAR2(100) NOT NULL,
   email VARCHAR2(80) UNIQUE,
   CONSTRAINT PK_USERS PRIMARY KEY(Id)
);
```

The Movies Database

```
CREATE TABLE COMMENTS (
  Id NUMBER,
  text CLOB NOT NULL,
  Movie NUMBER,
  UserId NUMBER,
  comment_date DATE,
  CONSTRAINT PK_COMMENTS PRIMARY KEY(Id)
);

CREATE TABLE RATINGS (
   UserId NUMBER,
   Movie NUMBER,
   Vote INTEGER CHECK(Vote>=0 AND Vote<=5),
   CONSTRAINT PK_RATINGS PRIMARY KEY(UserId,Movie)
);
```

The Movies Database

```
ALTER TABLE MOVIES ADD CONSTRAINT FK_MOVIES_GENRES
FOREIGN KEY(Genre) REFERENCES GENRES(Id) ON DELETE SET NULL;

ALTER TABLE MOVIES ADD CONSTRAINT FK_MOVIES_DIRECTORS
FOREIGN KEY(Director) REFERENCES DIRECTORS(Id);

ALTER TABLE MOVIES ADD CONSTRAINT FK_MOVIES_PRODUCTIONS
FOREIGN KEY(Production) REFERENCES PRODUCTIONS(Id);

ALTER TABLE COMMENTS ADD CONSTRAINT FK_COMMENTS_MOVIES
FOREIGN KEY(Movie) REFERENCES MOVIES(Code) ON DELETE CASCADE;

ALTER TABLE COMMENTS ADD CONSTRAINT FK_COMMENTS_USERS
FOREIGN KEY(UserId) REFERENCES USERS(Id) ON DELETE SET NULL;

ALTER TABLE RATINGS ADD CONSTRAINT FK_RATINGS_USERS
FOREIGN KEY(UserId) REFERENCES USERS(Id) ON DELETE SET NULL;

ALTER TABLE RATINGS ADD CONSTRAINT FK_RATINGS_MOVIES
FOREIGN KEY(Movie) REFERENCES MOVIES(Code) ON DELETE CASCADE;

ALTER TABLE MOVIE_CAST ADD CONSTRAINT FK_MOVIE_CAST_MOVIES
FOREIGN KEY(Movie) REFERENCES MOVIES(Code) ON DELETE CASCADE;

ALTER TABLE MOVIE_CAST ADD CONSTRAINT FK_MOVIE_CAST_ACTORS
FOREIGN KEY(Actor) REFERENCES ACTORS(Id);
```

Now try these exercises on your own!

1. **Create** the **tables** MOVIES(Code, Title, Description, Year, Director:DIRECTORS, Genre:GENRES, Duration), GENRES(Id, Name), and DIRECTORS(Id, Name, Surname) with the correct **intra-relational integrity constraints**.
2. **Modify** the **tables** to add the correct **foreign key constraints**.
3. **Insert** the following two **movies**:
   - ❑ **"Oppenheimer"** (2023, thriller, 180 minutes, directed by Christopher Nolan)
   - ❑ **"Barbie"** (2023, comedy, 114 minutes, directed by Greta Gerwig).

   Pay attention to the order of insertion into the tables.

Solution Ex.1

```
CREATE TABLE GENRES (
   Id NUMBER,
   Name VARCHAR2(100) NOT NULL,
   CONSTRAINT PK_GENRES PRIMARY KEY (Id)
);

CREATE TABLE DIRECTORS (
   Id NUMBER,
   Name VARCHAR2(100) NOT NULL,
   Surname VARCHAR2(100) NOT NULL,
   CONSTRAINT PK_DIRECTORS PRIMARY KEY (Id)
);
```

Solution Ex.1

```
CREATE TABLE MOVIES (
   Code NUMBER,
   Title VARCHAR2(500) NOT NULL,
   Description CLOB,
   Year NUMBER NOT NULL,
   Director NUMBER,
   Genre NUMBER,
   Duration NUMBER,
   CONSTRAINT PK_MOVIES PRIMARY KEY (Code),
   CONSTRAINT CK_MOVIES_YEAR CHECK (Year > 1900),
   CONSTRAINT CK_MOVIES_DURATION CHECK (Duration > 0)
);
```

Solution Ex.2

```
ALTER TABLE MOVIES
ADD CONSTRAINT FK_MOVIES_DIRECTORS
FOREIGN KEY (Director) REFERENCES DIRECTORS(Id);

ALTER TABLE MOVIES
ADD CONSTRAINT FK_MOVIES_GENRES
FOREIGN KEY (Genre) REFERENCES GENRES(Id);
```

Solution Ex.3

```
-- Insert genres
INSERT INTO GENRES VALUES (1, 'Thriller');

INSERT INTO GENRES VALUES (2, 'Comedy');

-- Insert directors
INSERT INTO DIRECTORS VALUES (1, 'Christopher', 'Nolan');

INSERT INTO DIRECTORS VALUES (2, 'Greta', 'Gerwig');

-- Insert movies
INSERT INTO MOVIES
VALUES (1, 'Oppenheimer', NULL, 2023, 1, 1, 180);

INSERT INTO MOVIES
VALUES (2, 'Barbie', NULL, 2023, 2, 2, 114);
```